Lab Sheet: 1

Objective:

- To design ER diagrams and convert entities and relationships into relational tables using proper schema notation.

Instructions:

1. Open your preferred Database Management System (DBMS) tool.
2. Create a new database named DBMS_YourName.
3. Draw ER diagrams for the given scenarios.
4. Convert entities and relationships into relational schemas.
5. Create tables using the derived schemas.

Relational Schema

PART A – COLLEGE DATABASE

STUDENT (USN, SName, Address, Phone, Gender)
SEMSEC (SSID, Sem, Sec)
CLASS (USN, SSID)
SUBJECT (Subcode, Title, Sem, Credits)
IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

PART B – COMPANY DATABASE

EMPLOYEE (SSN, Name, Address, Sex, Salary, Superssn, Dno)
DEPARTMENT (Dno, Dname, Mgrssn, Mgrstartdate)
DLOCATION (Dno, Dloc)
PROJECT (PNo, PName, PLocation, DNo)
WORKS_ON (SSN, PNo, Hours)

PART C – CUSTOMER SALE SCHEMA

- Database Schema for a customer-sale scenario
- Customer(Cust id : integer, cust_name: string)
- Item(item_id: integer, item_name:string, price: integer)
  Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

DDL Questions (Data Definition Language):

1. Write SQL statements to create all the above relations with appropriate PRIMARY KEY and FOREIGN KEY constraints.
2. Add a new column Email to the STUDENT relation.
3. Apply NOT NULL constraints to Salary and Price attributes.
4. Rename attribute SName to StudentName in STUDENT relation.
5. Drop the DLOCATION relation.

DML Questions (Data Manipulation Language):

1. Insert at least 10 tuples into CUSTOMER, ITEM, and SALE relations.
2. List all bills of the current date with customer names and item names.
3. Display total bill amount using quantity and price.
4. List customers who purchased products priced above 200.
5. Count the number of products purchased by each customer.
6. List products purchased by customer having Cust_id = 5.
7. Create a VIEW showing bill_no, bill_date, cust_id, item_id, price, qty,sold, amount.
8. Create a VIEW showing daily sales date-wise for the last one week.

ER DIAGRAM:

PART A – COLLEGE DATABASE

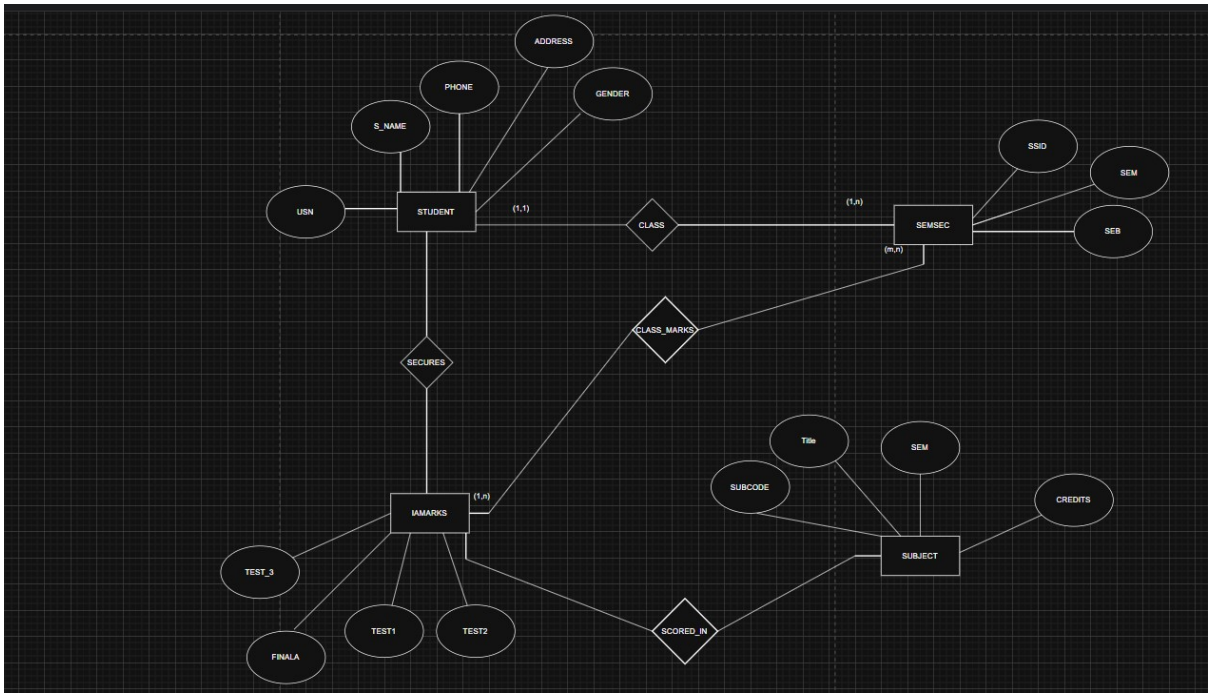
Entities and Attributes:

- **STUDENT**: USN (PK), SName, Address, Phone, Gender
- **SEMSEC**: SSID (PK), Sem, Sec
- **CLASS**: USN (FK → STUDENT), SSID (FK → SEMSEC)
- **SUBJECT**: Subcode (PK), Title, Sem, Credits
- **IAMARKS**: USN (FK → STUDENT), Subcode (FK → SUBJECT), SSID (FK → SEMSEC), Test1, Test2, Test3, FinalIA

Relationships:

- CLASS → maps STUDENT to SEMSEC
- IAMARKS → maps STUDENT, SUBJECT, SEMSEC with marks

ER Diagram (Text Representation)



PART B – COMPANY DATABASE

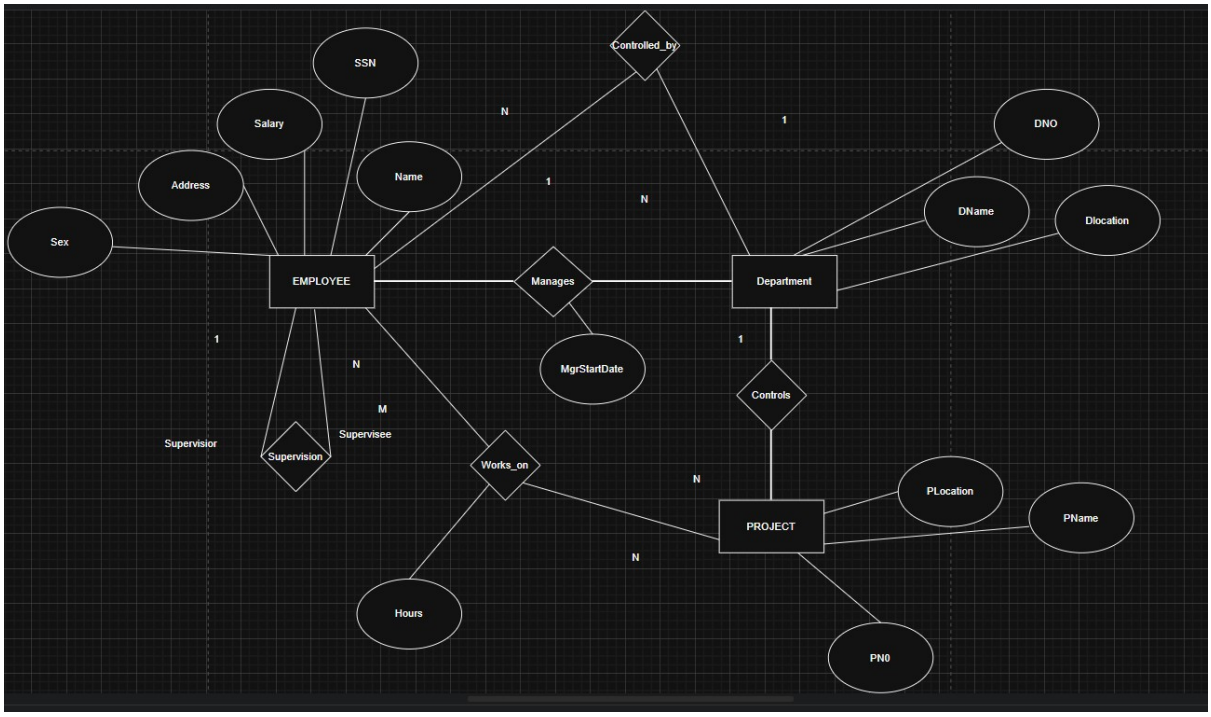
Entities and Attributes:

- **EMPLOYEE**: SSN (PK), Name, Address, Sex, Salary, Superssn (FK → EMPLOYEE), Dno (FK → DEPARTMENT)

- **DEPARTMENT**: Dno (PK), Dname, Mgrssn (FK → EMPLOYEE), Mgrstartdate
- **DLOCATION**: Dno (FK → DEPARTMENT), Dloc
- **PROJECT**: PNo (PK), PName, PLocation, DNo (FK → DEPARTMENT)
- **WORKS_ON**: SSN (FK → EMPLOYEE), PNo (FK → PROJECT), Hours

Relationships:

- EMPLOYEE works in DEPARTMENT
- DEPARTMENT manages PROJECT
- EMPLOYEE works_on PROJECT
- DEPARTMENT has DLOCATION



PART C – CUSTOMER SALE SCHEMA:

Entities and Attributes:

- CUSTOMER(Cust_id PK, cust_name)
- ITEM(Item_id PK, item_name, price)
- SALE(Bill_no PK, Bill_date, Cust_id → CUSTOMER, Item_id → ITEM, qty_sold)

Relational Schema:

- CUSTOMER(Cust_id, cust_name)
- ITEM(Item_id, item_name, price)
- SALE(Bill_no, Bill_date, Cust_id, Item_id, qty_sold)

DDL Questions (Data Definition Language)

Q1. Write SQL statements to create all the above relations with PRIMARY KEY and FOREIGN KEY constraints.

Code:

```
CREATE DATABASE DBMS_AakashYadav2026;
USE DBMS_AakashYadav2026;

CREATE TABLE STUDENT(
  USN VARCHAR(10) PRIMARY KEY,
  SName VARCHAR(50),
  Address VARCHAR(100),
  Phone VARCHAR(15),
  Gender CHAR(1)
);

CREATE TABLE SEMSEC(
  SSID INT PRIMARY KEY,
  Sem INT,
  Sec CHAR(1)
);

CREATE TABLE CLASS(
  USN VARCHAR(10),
  SSID INT,
  PRIMARY KEY(USN, SSID),
  FOREIGN KEY (USN) REFERENCES STUDENT(USN),
  FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);

CREATE TABLE SUBJECT(
  Subcode VARCHAR(10) PRIMARY KEY,
  Title VARCHAR(50),
  Sem INT,
  Credits INT
);

CREATE TABLE IAMARKS(
  USN VARCHAR(10),
  Subcode VARCHAR(10),
  SSID INT,
  Test1 INT,
  Test2 INT,
  Test3 INT,
  FinalIA INT,
  PRIMARY KEY(USN, Subcode, SSID),
```

```sql
  FOREIGN KEY (USN) REFERENCES STUDENT(USN),
  FOREIGN KEY (Subcode) REFERENCES SUBJECT(Subcode),
  FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);

CREATE TABLE DEPARTMENT(
  Dno INT PRIMARY KEY,
  Dname VARCHAR(50),
  Mgrssn CHAR(9),
  Mgrstartdate DATE
);

CREATE TABLE EMPLOYEE(
  SSN CHAR(9) PRIMARY KEY,
  Name VARCHAR(50),
  Address VARCHAR(100),
  Sex CHAR(1),
  Salary DECIMAL(10,2) NOT NULL,
  Superssn CHAR(9),
  Dno INT,
  FOREIGN KEY(Superssn) REFERENCES EMPLOYEE(SSN),
  FOREIGN KEY(Dno) REFERENCES DEPARTMENT(Dno)
);

ALTER TABLE DEPARTMENT
ADD CONSTRAINT fk_mgr
FOREIGN KEY(Mgrssn) REFERENCES EMPLOYEE(SSN);

CREATE TABLE PROJECT(
  PNo INT PRIMARY KEY,
  PName VARCHAR(50),
  PLocation VARCHAR(50),
  DNo INT,
  FOREIGN KEY(DNo) REFERENCES DEPARTMENT(Dno)
);

CREATE TABLE WORKS_ON(
  SSN CHAR(9),
  PNo INT,
  Hours DECIMAL(5,2),
  PRIMARY KEY(SSN, PNo),
  FOREIGN KEY(SSN) REFERENCES EMPLOYEE(SSN),
  FOREIGN KEY(PNo) REFERENCES PROJECT(PNo)
);

CREATE TABLE CUSTOMER(
  Cust_id INT PRIMARY KEY,
  Cust_name VARCHAR(50)
```

```
);

CREATE TABLE ITEM(
  Item_id INT PRIMARY KEY,
  Item_name VARCHAR(50),
  Price INT NOT NULL
);

CREATE TABLE SALE(
  Bill_no INT PRIMARY KEY,
  Bill_date DATE,
  Cust_id INT,
  Item_id INT,
  Qty_sold INT,
  FOREIGN KEY(Cust_id) REFERENCES CUSTOMER(Cust_id),
  FOREIGN KEY(Item_id) REFERENCES ITEM(Item_id)
);
```

Q2. Add a new column Email to STUDENT.

```
ALTER TABLE STUDENT ADD Email VARCHAR(50);
```

Q3: Apply NOT NULL constraints to Salary and Price attributes.

```
ALTER TABLE EMPLOYEE
MODIFY Salary DECIMAL(10,2) NOT NULL;

ALTER TABLE ITEM
MODIFY Price INT NOT NULL;
```

Q4. Rename attribute SName to StudentName in STUDENT relation.

```
ALTER TABLE STUDENT
CHANGE SName StudentName VARCHAR(50);
```

Q5. Drop the DLOCATION relation.

```
DROP TABLE DLOCATION;
```

DML QUESTIONS (CUSTOMER–SALE SCHEMA)

Q1. Insert at least 10 tuples into CUSTOMER, ITEM, and SALE relations.

```
INSERT INTO CUSTOMER VALUES
(1,'Aakash'),
(2,'Vivek'),
(3,'Rahul'),
(4,'Mayank'),
(5,'Karan'),
(6,'Ravi'),
(7,'Amit'),
(8,'Vikas'),
(9,'Rohit'),
(10,'Arjun');

INSERT INTO ITEM VALUES
(101,'Pen',10),
(102,'Book',250),
(103,'Bag',800),
(104,'Pencil',5),
(105,'Notebook',120),
(106,'Shoes',1500),
(107,'Bottle',200),
(108,'Scale',15),
(109,'Laptop',50000),
(110,'Mouse',600);

INSERT INTO SALE VALUES
(1,CURDATE(),1,101,5),
(2,CURDATE(),2,102,2),
(3,CURDATE(),3,103,1),
(4,CURDATE(),4,105,3),
(5,CURDATE(),5,106,1),
(6,CURDATE(),6,110,2),
(7,CURDATE(),7,107,4),
(8,CURDATE(),8,102,1),
(9,CURDATE(),9,103,2),
(10,CURDATE(),10,109,1);
```

Q2. List all bills of the current date with customer names and item names.

```
SELECT s.Bill_no, c.Cust_name, i.Item_name
FROM SALE s
JOIN CUSTOMER c ON s.Cust_id = c.Cust_id
JOIN ITEM i ON s.Item_id = i.Item_id
WHERE s.Bill_date = CURDATE();
```

Q3. Display total bill amount using quantity and price.

```
SELECT s.Bill_no,
       SUM(s.Qty_sold * i.Price) AS Total_Amount
FROM SALE s
JOIN ITEM i ON s.Item_id = i.Item_id
GROUP BY s.Bill_no;
```

Q4. List customers who purchased products priced above 200.

```
SELECT DISTINCT c.Cust_name
FROM CUSTOMER c
JOIN SALE s ON c.Cust_id = s.Cust_id
JOIN ITEM i ON s.Item_id = i.Item_id
WHERE i.Price > 200;
```

Q5. Count the number of products purchased by each customer.

```
SELECT c.Cust_name,
       SUM(s.Qty_sold) AS Total_Products
FROM CUSTOMER c
JOIN SALE s ON c.Cust_id = s.Cust_id
GROUP BY c.Cust_name;
```

Q6. List products purchased by customer having Cust_id = 5.

```
SELECT i.Item_name
FROM SALE s
JOIN ITEM i ON s.Item_id = i.Item_id
WHERE s.Cust_id = 5;
```

Q7. Create a VIEW showing bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```
CREATE VIEW BILL_DETAILS AS
SELECT s.Bill_no,
       s.Bill_date,
       s.Cust_id,
       s.Item_id,
       i.Price,
       s.Qty_sold,
       (s.Qty_sold * i.Price) AS Amount
FROM SALE s
JOIN ITEM i ON s.Item_id = i.Item_id;
```

Q8. Create a VIEW showing daily sales date-wise for the last one week.

```
CREATE VIEW WEEKLY_SALES AS
SELECT s.Bill_date,
       SUM(s.Qty_sold * i.Price) AS Daily_Total
```

```
FROM SALE s
JOIN ITEM i ON s.Item_id = i.Item_id
WHERE s.Bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY s.Bill_date;
```